

Challenge Final 2026

Get Talent IA | De una necesidad real a una solución GenAI evaluable, consumible y responsable

Fecha límite: miércoles 30 de septiembre de 2026 - 23:59 h

Objetivos

Integrar los conocimientos adquiridos durante el programa mediante una **solución de Inteligencia Artificial de punta a punta** aplicada a un problema real.

- Definir un problema y justificar el uso de IA.
- Preparar documentación y construir una arquitectura RAG.
- Aplicar chunking, embeddings, búsqueda semántica y retrieval.
- Diseñar prompts efectivos.
- Evaluar retrieval y generation.
- Exponer la solución con FastAPI.
- Incorporar IA Responsable y Human in the Loop.
- Documentar decisiones técnicas.

Arquitectura mínima esperada

Documentos -> limpieza / chunking / metadata -> embeddings -> vector store -> retrieval -> contexto -> prompt -> LLM -> respuesta -> evaluación -> FastAPI

Además: incorporar un punto de control humano coherente con el riesgo del caso de uso.

1. Definición del problema

Seleccionar un problema concreto que pueda abordarse mediante IA. Puede relacionarse con documentación empresarial, procedimientos, soporte, onboarding, búsqueda de conocimiento, análisis documental u otro dominio de interés.

Explicar claramente: **Problema -> Usuario -> Necesidad -> Solución propuesta**. Debe existir una justificación clara para utilizar IA y una arquitectura RAG.

2. Recopilación y preparación de datos

Seleccionar documentos relacionados con el problema. Como mínimo: extracción de texto, limpieza cuando sea necesaria, **chunking y metadata**. Justificar brevemente la estrategia de chunking.

3. Embeddings y Vector Store

Transformar los chunks en embeddings y almacenarlos en una base o índice vectorial. Puede utilizarse ChromaDB u otra tecnología equivalente. Conservar metadata útil como fuente, página, categoría, fecha o versión cuando corresponda.

4. Módulo de Retrieval

Recuperar información relevante a partir de una pregunta. Definir y justificar estrategia de búsqueda, Top-K y uso de metadata. El retrieval debe poder inspeccionarse independientemente de la generación.

5. Prompting + Generación

Construir un System Prompt que defina rol, uso de evidencia, restricciones, formato esperado y comportamiento cuando no exista información suficiente.

El sistema no debe inventar información para completar respuestas cuando no exista evidencia suficiente.

6. Evaluacion del RAG

Construir un dataset de evaluacion con **un minimo de 5 preguntas**: preguntas respondibles, preguntas que exijan recuperar informacion especifica y al menos una cuya respuesta no este en la documentacion.

Retrieval: ¿se recupero la evidencia necesaria?

Generation: ¿la respuesta es correcta, relevante y respaldada por el contexto?

Se puede utilizar rubrica manual, metricas, LLM-as-a-Judge o una combinacion. Explicar que se evaluo, como y que se aprendio.

7. API REST con FastAPI

La solucion debe exponerse mediante API REST. Endpoints minimos:

```
GET /health
POST /retrieve
POST /ask
```

/retrieve permite inspeccionar chunks. **/ask** ejecuta pregunta -> retrieval -> contexto -> prompt -> LLM -> respuesta + fuentes. La API debe documentarse con Swagger y poder probarse desde Swagger o Postman.

8. IA Responsable + Human in the Loop

Requisito obligatorio. Identificar al menos una situacion en la que sea necesaria intervencion humana antes de continuar con una accion o decision.

IA analiza / recomienda -> evalua si requiere supervision -> revision humana -> aprobar o rechazar -> continuar o detener.

La implementacion puede usar confirmacion, estado **pending_approval**, endpoint adicional, flujo de aprobacion o simulacion funcional equivalente.

Explicar: **que puede hacer la IA automaticamente, que requiere intervencion humana y que riesgo se mitiga.**

9. Pruebas

Demostrar funcionamiento de punta a punta. Probar al menos: pregunta valida, consulta sin evidencia suficiente, recuperacion de fuentes, respuesta del LLM, Human in the Loop y un error controlado.

10. Extensiones optativas - Semana 5

Los contenidos de la ultima semana son **bonus de innovacion** y no reemplazan requisitos obligatorios.

- **Copilot Studio:** agente como interfaz o capa de interaccion.
- **n8n:** workflow que consuma la API y ejecute acciones.
- **Microsoft Foundry:** modelos/agentes o evaluacion y observabilidad.
- **Agents + Tools/Skills:** agente que seleccione y utilice herramientas.
- Otras: reranking, filtros de metadata, historial, GUI, tracing u otra mejora justificada.

Criterios de evaluacion

Dimension	Peso
Problema y diseno	10%
Datos + Chunking + Embeddings	15%
Retrieval + Vector Store	15%
Prompting + Generacion	15%

Dimension	Peso
Evaluacion del RAG	15%
FastAPI + end-to-end	15%
IA Responsable + Human in the Loop	10%
Documentacion + presentacion	5%
TOTAL	100%

Las extensiones de Semana 5 son valor agregado, pero no compensan la ausencia de componentes obligatorios.

Formato de entrega

Fecha limite: miercoles 30 de septiembre de 2026 a las 23:59 h.

Codigo fuente: repositorio Git o archivo comprimido. Incluir codigo, requirements.txt, README.md, .env.example, documentacion y documentos/dataset o instrucciones para obtenerlos.

Seguridad: no incluir API Keys, tokens, contraseñas ni archivos .env con credenciales reales.

Reproducibilidad: incluir todo lo necesario para ejecutar el proyecto en otra computadora.

README: problema, arquitectura, instalacion, ejecucion, endpoints, estrategia RAG, evaluacion y Human in the Loop.

Presentacion: Problema -> Solucion -> Arquitectura -> Demo -> Evaluacion -> IA Responsable/HITL -> Conclusiones.

Demostracion: la solucion debe estar operativa de punta a punta. Podran solicitarse consultas adicionales durante la presentacion.

El objetivo no es solamente que el modelo responda: es demostrar que pueden construir, evaluar, exponer y controlar una solucion GenAI completa.